

Chiarissimo, ora ho capito esattamente cosa intendi: vuoi il quadro completo e operativo ("la distinta base" sia economica che tecnica) per trasformare questo prototipo in un sistema pronto a tutto, senza sorprese in fattura e senza codice scritto a metà.

Ecco la guida definitiva per configurare Azure (con i costi stimati nel 2026) e le attività di sviluppo da fare sul tuo PC.

1. Come configurare Azure (Infrastruttura, Sicurezza e Costi)

Per non far lievitare i costi, sfruttiamo i servizi in modalità **Serverless/PaaS**. Evitiamo configurazioni enterprise inutili (come i *Private Endpoints* che costano circa 10-15€ al mese l'uno) e usiamo i **Service Endpoints** inclusi nella VNet, che sono **completamente gratuiti**.

La lista dei componenti e il bilancio mensile:

Risorsa su Azure	Configurazione da impostare	Costo Mensile Stimato
Azure IoT Hub	Livello B1 (Basic) o F1 (Free) . Per i test, il piano Free gestisce fino a 8.000 messaggi al giorno. Se passi a produzione, il piano B1 gestisce fino a 400.000 messaggi al giorno.	0 € (Free) / ~9,30 € (B1)
Azure Function	Piano Flex Consumption . Configura l'istanza con la memoria minima (2048 MB) e attiva l' Identity (System Assigned: On) .	~0 € (Primi 400.000 GB/s gratis, poi paghi solo al millisecondo di uso reale)
Azure Cosmos DB	Modalità Serverless (fondamentale: non scegliere "Provisioned Throughput"). Paghi solo per i gigabyte occupati e per le operazioni di lettura/scrittura (RU).	~0,23 € per GB + frazioni di centesimo per ogni scrittura.
Virtual Network (VNet)	Crea una VNet con una subnet. Abilita i Service	0 € (I Service Endpoints e le VNet standard sono gratuiti)

	Endpoints per Microsoft.AzureCosmosDB e Microsoft.Storage dentro la subnet.	
Azure Storage Account	Tipo standard (LRS). Serve alla Function per esistere.	~1,00 - 2,00 € (In base ai file di log memorizzati)
TOTALE STIMATO	<i>Ambiente blindato, scalabile e sicuro</i>	Da 2,00 € a 12,00 € / mese

Azioni di Sicurezza sul Portale:

1. **Asgnazione Ruoli (RBAC):** Vai su Cosmos DB e IoT Hub, clicca su *Access Control (IAM)* e dai i permessi di scrittura/lettura all'identità della Function (Managed Identity), come descritto prima.
2. **Isolamento di Rete:** Nelle impostazioni di Firewall e Reti di Cosmos DB e dello Storage Account, seleziona "Consenti accesso solo da reti selezionate" e inserisci la subnet della tua Function.

2. Cosa devi sviluppare In-House (Visual Studio e C#)

Il tuo codice attuale funziona, ma per accettare la nuova architettura senza password e per essere mantenibile, devi fare questi passi sul tuo PC:

A. Aggiornare i Pacchetti NuGet

Devi installare le versioni più recenti delle estensioni che supportano l'autenticazione tramite Token (senza password):

- Azure.Identity (permette a .NET di capire chi sei tu in locale e chi è la Function su Azure).
- Microsoft.Azure.Functions.Worker.Extensions.CosmosDB (versione 4.x o superiore).
- Microsoft.Azure.Functions.Worker.Extensions.EventHubs (versione 6.x o superiore).

B. Modificare il codice C# (I Trigger senza password)

Il codice deve smettere di cercare le stringhe con le chiavi primarie, ma deve puntare ai **Fully Qualified Namespace** (gli indirizzi puliti).

La firma della tua Function diventerà così:

```

C#
Function("SalvaMisureIoT")
CosmosDBOutput{
    databaseName: "IoTDatabase",
    containerName: "Misure",
    Connection = "CosmosDBConnection") // Punterà solo all'URL dell'endpoint
public object Run
    EventHubTrigger(
        "%IoTHubName%",
        Connection = "IoTHubConnection", // Punterà all'indirizzo dell'Event Hub, senza chiavi
        ConsumerGroup = "$Default")
    string messages
    FunctionContext context

```

C. Configurare il local.settings.json "In Casa"

Per fare in modo che il tuo PC riesca a fare il debug, nel file locale userai gli indirizzi puliti. Il sistema userà il tuo login di Visual Studio per autenticarsi temporaneamente su Azure durante il debug:

```

JSON
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated"

    // Solo l'endpoint, senza AccountKey!
    "CosmosDBConnection__accountEndpoint": "https://db-gigi-iot.documents.azure.com:443/"

    // Solo il namespace dell'Event Hub integrato, senza SharedAccessKey!
    "IoTHubConnection__fullyQualifiedNamespace":
    "ihsuprdbnres002dednamespace.servicebus.windows.net"

    "IoTHubName": "iothub-ehub-iotgigi1-67663922-d98aa7a5b1"
  }
}

```

(Nota il doppio trattino basso __ nel file JSON: serve a .NET per mappare le sotto-proprietà dell'autenticazione moderna).

D. Gestione dell'errore e stabilità nel codice

Nel blocco di codice C# dove cicli i messaggi, devi aggiungere un sistema di **Retry Policy** o un blocco try-catch robusto. Se un messaggio JSON arriva corrotto dai sensori, non deve mandare in crash l'intera Function (altrimenti blocchi la lettura dei messaggi successivi provenienti dall'IoT Hub).

Con questa "lista della spesa", l'ambiente diventa professionale, strutturato e con costi ridotti all'osso.